

Industrial Internship Report on “Voice Calculator”

Prepared by
Vaibhav Chormar

Executive Summary

This report presents the details of the Industrial Training undergone at UniConverge Technologies Pvt. Ltd. (UCT), Noida, as part of the curriculum of Jaipur Engineering College and Research Centre (JECRC), Jaipur, under the guidance of Dr. Purnima Sharma, ITS Coordinator.

The training was focused on a project assigned by UCT, which had to be completed along with this report within the given training period.

My project was the design and development of a Voice Calculator using Python — a desktop application that listens to a spoken mathematical expression through a microphone, converts the speech into text, calculates the result, and presents the answer both on screen and through voice output.

This training gave me a very good opportunity to get exposure to a real, industry-style problem and to design and implement a working solution for it. It was an overall great learning experience.

TABLE OF CONTENTS

1	Preface	Error! Bookmark not defined.
2	Introduction	Error! Bookmark not defined.
2.1	About UniConverge Technologies Pvt Ltd	Error! Bookmark not defined.
2.2	About upskill Campus	Error! Bookmark not defined.
2.3	Objective	Error! Bookmark not defined.
2.4	Reference	Error! Bookmark not defined.
2.5	Glossary.....	Error! Bookmark not defined.
3	Problem Statement.....	Error! Bookmark not defined.
4	Existing and Proposed solution.....	Error! Bookmark not defined.
5	Proposed Design/ Model	Error! Bookmark not defined.
5.1	High Level Diagram (if applicable)	Error! Bookmark not defined.
5.2	Low Level Diagram (if applicable)	Error! Bookmark not defined.
5.3	Interfaces (if applicable)	Error! Bookmark not defined.
6	Performance Test.....	Error! Bookmark not defined.
6.1	Test Plan/ Test Cases	Error! Bookmark not defined.
6.2	Test Procedure	Error! Bookmark not defined.
6.3	Performance Outcome	Error! Bookmark not defined.
7	My learnings.....	Error! Bookmark not defined.
8	Future work scope	Error! Bookmark not defined.

1. Preface

This report summarizes the work carried out during my Industrial Training at UniConverge Technologies Pvt. Ltd. (UCT). Industrial training is an important part of an engineering career because it bridges the gap between classroom learning and real-world software development. It gives students the chance to work on a genuine problem, follow a proper development process, and get a sense of how an engineering team plans, builds, and tests a product.

My project during this training was the Voice Calculator using Python, an application that lets a user perform calculations by speaking instead of typing. The idea, the design, and the implementation are all covered in detail in the following sections.

JECRC, along with the training coordinator Dr. Purnima Sharma, and UniConverge Technologies (UCT) together made this opportunity possible by providing access to the problem statement, the required guidance, and a structured training period to plan, build, and document the project.

The training was planned in stages — understanding the problem, learning the required Python libraries (SpeechRecognition, pyttsx3, Tkinter, word2number), building the interface, building the backend logic, testing the application with different voice commands, and finally preparing this report and the project presentation.

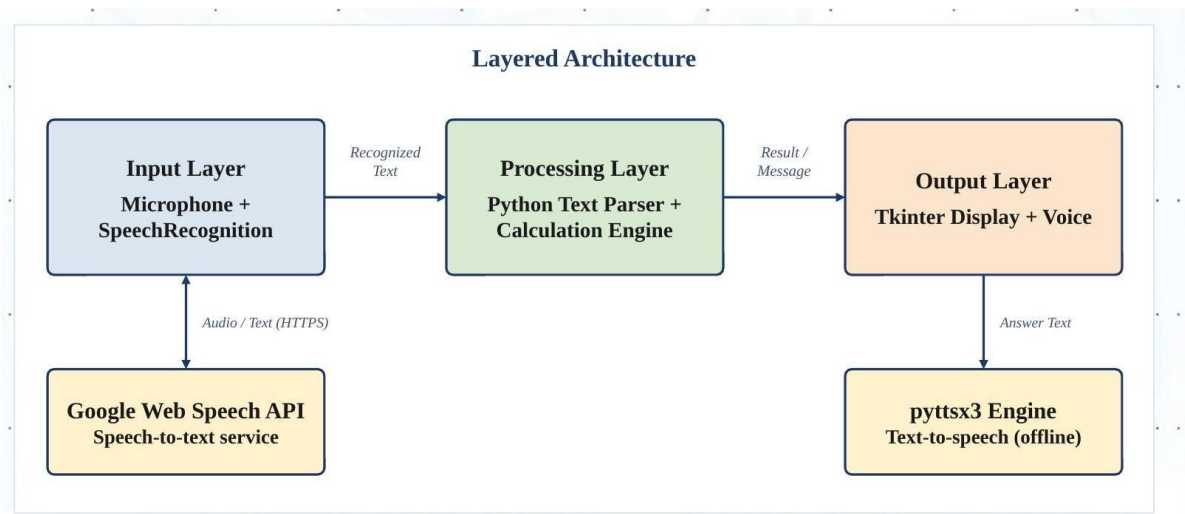


Figure: Overview of the Voice Calculator system built during the training

Through this training, I learned how to combine several Python libraries into one working application, how to handle real-world issues like unclear speech or a missing internet connection, and how to keep code safe by avoiding risky functions such as `eval()`. It also improved my confidence in presenting technical work clearly.

I sincerely thank Dr. Purnima Sharma for her constant guidance and support throughout this training, and UniConverge Technologies Pvt. Ltd. for providing this valuable opportunity.

To my juniors and peers who will take up similar training in the future: pick a project you are genuinely curious about, start with a simple working version before adding features, and test your application with real, sometimes messy, input early — that is where most of the learning happens.

1 2. Introduction

1.1 2.1 About UniConverge Technologies Pvt. Ltd. (UCT)

UniConverge Technologies Pvt. Ltd. (UCT) is a company established in 2013 by alumni of IIT and NIT. It works in the Industrial IoT domain, specializing in wireless communication, LoRaWAN networks, remote monitoring, and smart automation solutions for industries such as manufacturing, energy, and logistics.

Established	2013, by IIT/NIT alumni
Headquarters	11 Ground Floor, Sector 62, Noida, Uttar Pradesh, India
Core Domain	Industrial IoT, LoRaWAN wireless communication, remote monitoring & automation
Mission	To empower businesses with innovative, customized IoT solutions built on trust, transparency, and long-term partnerships
Key Clients	ISRO, HAL, Tata Motors, NTPC, and IOCL
Scale	1,100+ clients served and 700+ projects delivered

1.2 2.2 About the Industrial Training Program

This Industrial Training was carried out as part of the academic curriculum of Jaipur Engineering College and Research Centre (JECRC), under the overall coordination of Dr. Purnima Sharma (ITS Coordinator), in association with UniConverge Technologies Pvt. Ltd. (UCT). The program gave students a real problem statement to work on, along with the freedom to design and build their own solution, followed by evaluation through a report and a final presentation.

1.3 2.3 Objectives of the Training

The objectives of this Industrial Training program were to:

- get practical, hands-on experience of working on an industry-style problem.
- learn to design and build a complete, working software solution from scratch.
- improve understanding of Python and its application libraries beyond classroom syllabus.

- develop better problem solving, debugging, and technical communication skills.
- build confidence in presenting and documenting a technical project professionally.

1.4 2.4 Reference

The following references and official documentation were used while building the Voice Calculator:

- SpeechRecognition Library — <https://pypi.org/project/SpeechRecognition/>
- pyttsx3 Documentation — <https://pyttsx3.readthedocs.io/>
- Python Tkinter Documentation — <https://docs.python.org/3/library/tkinter.html>
- Python ast Module Documentation — <https://docs.python.org/3/library/ast.html>
- word2number Library — <https://pypi.org/project/word2number/>

1.5 2.5 Glossary

Term	Meaning
GUI	Graphical User Interface — the visual window a user interacts with
API	Application Programming Interface — a way for two programs to talk to each other
STT	Speech-to-Text — converting spoken audio into written text
TTS	Text-to-Speech — converting written text into spoken audio
DFD	Data Flow Diagram — a diagram showing how data moves through a system
ast	Abstract Syntax Tree — a safe way in Python to read and evaluate expressions

2 3. Problem Statement

Typing numbers and operators on a keyboard or an on-screen calculator is slow, and it is not always convenient — for example, when a user's hands are busy, or for users with limited mobility or visual difficulty. A more natural and accessible way to perform quick calculations is needed.

The assigned problem statement was: design and implement a Voice Calculator in Python that listens to a spoken mathematical expression, converts it to text, calculates the answer accurately, and presents the result both visually on screen and audibly through speech, while handling common errors gracefully.

3 4. Existing and Proposed Solution

Existing solutions largely fall into two categories. The first is the standard on-screen or physical calculator, which requires the user to type every number and operator manually — this is fast for an experienced user but inconvenient when hands-free operation is needed. The second is the voice assistant built into smartphones (such as Google Assistant or Siri), which can answer simple spoken calculations but works only as a small part of a much larger, closed system, is not customizable, and always depends on constant internet access and a proprietary backend.

The solution proposed and built in this project is a lightweight, standalone Voice Calculator written entirely in Python. It listens for a spoken expression, uses the SpeechRecognition library with the Google Web Speech API to convert speech to text, parses number and operator words into a valid mathematical expression using the word2number library, safely evaluates the expression using Python's ast module, and finally shows the result on a simple Tkinter window while also speaking it aloud using the offline pyttsx3 engine.

The value addition of this project is that it is open, lightweight, and fully explainable — every step from listening to speaking the answer is visible and can be modified, unlike a closed voice-assistant ecosystem. It also adds safety (using ast instead of eval), a history of past calculations, and support for advanced operations such as powers, roots, remainders, and percentages, in addition to basic arithmetic.

4 5. Proposed Design / Model

This section explains the design of the Voice Calculator — its layered architecture, its data flow, and its user interface — covering the flow from the initial voice input to the final spoken output.

4.1 5.1 High Level Diagram — System Architecture

The system follows a layered architecture that clearly separates voice input, processing, and output, as shown below.

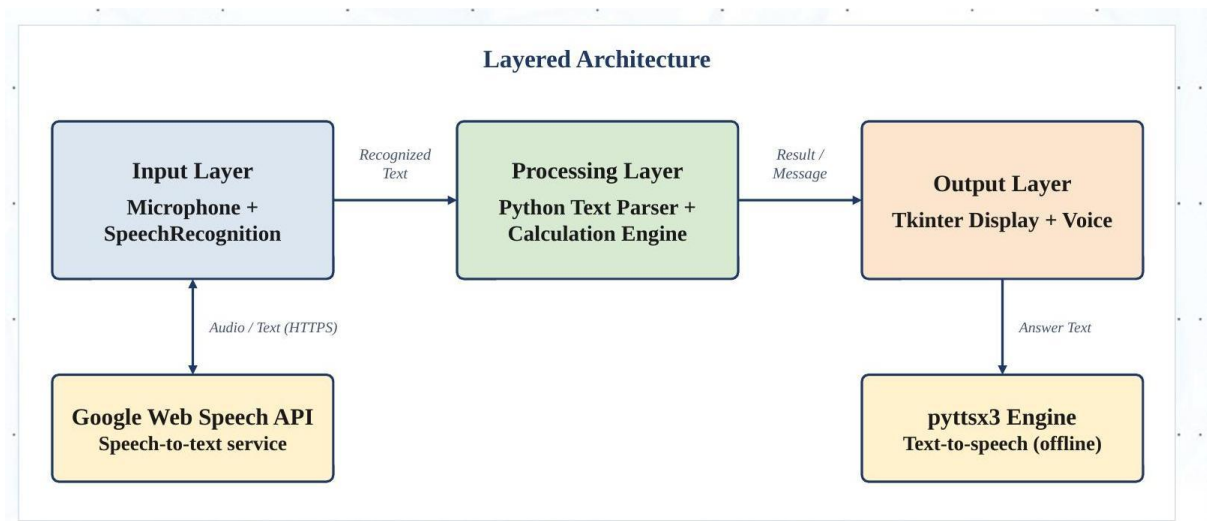


Figure 1: High-level layered architecture of the Voice Calculator

The Input Layer captures audio through the microphone and uses the SpeechRecognition library, which calls the Google Web Speech API over HTTPS to convert the audio into recognized text. The Processing Layer, built entirely in Python, parses this text into a valid mathematical expression and safely calculates the result. The Output Layer displays the result and any messages in the Tkinter window and also speaks the answer aloud using the pyttsx3 engine, which works completely offline.

4.2 5.2 Low Level Diagram — Data Flow Diagram (DFD)

The Data Flow Diagram below is shown at two levels. Level 0 (the context diagram) shows the Voice Calculator as a single process interacting with two external entities — the User and the Speech Recognition Service. Level 1 breaks this single process down into four core processes and two data stores.

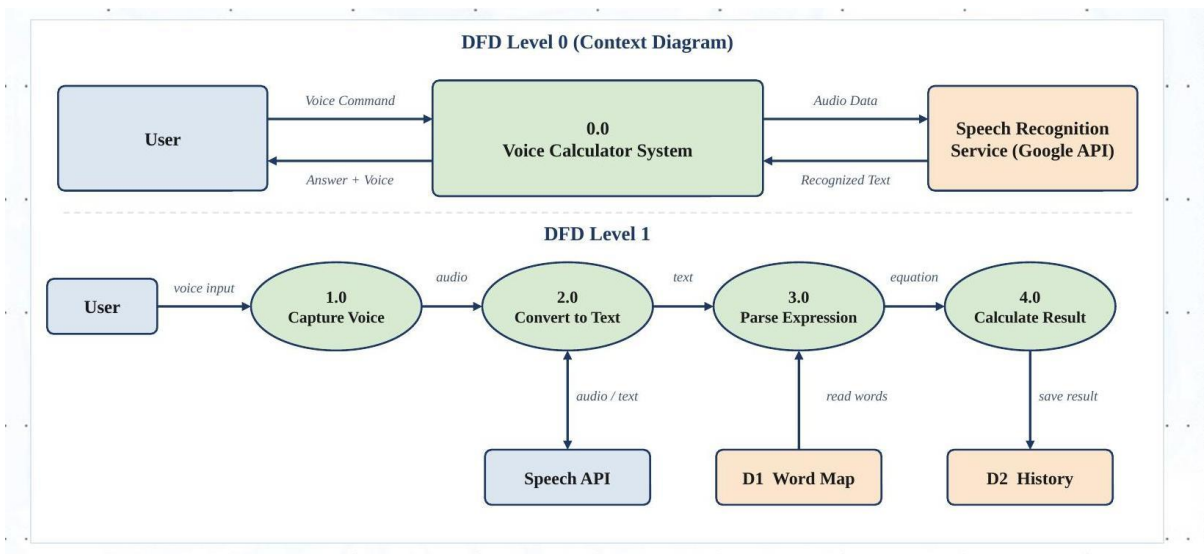


Figure 2: Data Flow Diagram (Level 0 and Level 1) of the Voice Calculator

In Level 1, process 1.0 (Capture Voice) records the audio; process 2.0 (Convert to Text) sends it to the Speech API and gets back recognized text; process 3.0 (Parse Expression) reads number and operator words from the D1 Word Map data store and converts them into a valid expression; and process 4.0 (Calculate Result) evaluates the expression and saves the outcome to the D2 History data store before it is returned to the user.

4.3 5.3 Interfaces — User Interface and Workflow

The user interface is built with Tkinter, Python's built-in GUI toolkit, as a single, simple window. It has a large mic button with a live listening status, a display panel that shows the recognized text, the parsed expression, and the final answer, and voice output that speaks the result aloud. The key screens implemented are the Main Calculator Window, History Panel, Listening Indicator, Error Messages, and Result Display.

The application workflow, from launch to spoken result, is as follows:

1. Start / Launch the application.
2. Listen: the user clicks the mic button and speaks an expression, which is converted to text by SpeechRecognition.
3. Parse: if the speech is understood, number and operator words are converted into a valid math expression; if not, the app asks the user to repeat.
4. Calculate: the expression is safely evaluated using Python's ast module to get the result.
5. Respond: the answer (or a friendly error message) is displayed on screen and spoken aloud, and saved to the calculation history before the app waits for the next command.

The application supports three groups of voice commands — Basic (for example “5 plus 3” = 8, “6 times 7” = 42), Advanced (for example “2 power 3” = 8, “square root of 16” = 4, “20 percent of 50” = 10), and Control commands (“clear”, “history”, “repeat”, and “exit”).

5 6. Performance Test

Since a voice-driven application depends on external factors such as microphone quality, background noise, and internet speed, testing was an important part of this project to make sure the application behaves reliably under normal use, and fails gracefully otherwise.

5.1 6.1 Test Plan / Test Cases

Test Case	Expected Result
Speak a basic expression, e.g. "5 plus 3"	Displays and speaks "8"
Speak an advanced expression, e.g. "square root of 16"	Displays and speaks "4"
Speak "8 divided by 0"	Shows a friendly divide-by-zero error message instead of crashing
Speak unclear or mumbled audio	App asks the user to repeat instead of giving a wrong answer
Disconnect the internet and try to speak a command	App shows a "no internet connection" message
Say "history" after a few calculations	Displays the list of past calculations
Say "repeat"	Speaks the last answer again without recalculating

5.2 6.2 Test Procedure

Each test case was executed manually by running the application, clicking the mic button, and speaking the required command in a normal room environment. The recognized text, the parsed expression, the displayed result, and the spoken output were each checked against the expected result. Edge cases — such as divide by zero, unrecognized speech, and no internet connection — were tested separately by deliberately triggering those conditions.

5.3 6.3 Performance Outcome

The Voice Calculator correctly handled all the basic and advanced arithmetic commands listed in the test plan, and produced accurate results in each case. Error handling worked as intended — unclear speech prompted a retry instead of a wrong calculation, divide-by-zero showed a friendly message instead of crashing the app, and a missing internet connection was detected and reported clearly, since the Google Web Speech API requires an active connection while the pyttsx3 voice output continued to work offline. Overall, the application performed reliably for everyday arithmetic and the supported advanced operations.

6 7. My Learnings

This training gave me practical experience of building a complete application rather than just an isolated script. I learned how to combine multiple Python libraries — SpeechRecognition, pyttsx3, Tkinter, and word2number — into one working system, and how to design a layered architecture that keeps input, processing, and output cleanly separated.

I also learned the importance of safe coding practices, such as using Python's `ast` module instead of `eval()` to evaluate expressions, and the value of proper error handling for real-world issues like unclear audio or a lost internet connection. Beyond the technical skills, this training improved my ability to plan a project in stages, document my work clearly, and present it confidently — skills that I believe will help me throughout my engineering career.

7 8. Future Work Scope

Due to the limited time available during this training, a few ideas could not be implemented but can be taken up in the future:

- Adding support for offline speech recognition, so the calculator can work without an internet connection.
- Supporting multi-step and multi-language voice commands, including regional languages.
- Adding a scientific mode with trigonometric and logarithmic functions.
- Packaging the application into a standalone installable file for Windows and Linux.
- Building a mobile version of the Voice Calculator using a framework such as Kivy.